TestSplit User Manual

Authors: Samson Oloruntola (22714745) & Marjia Siddik (22306501)
Supervisor: Dr Paul Clarke
Module: CSC1097 - Final Year Project
Submission Date: 01/05/2026

Table of Contents

1. Motivation

1.1 Purpose of the Manual

This user manual provides guidance on installing, configuring, and using TestSplit, a tool designed to optimise Continuous Integration (CI) pipelines by parallelising test distribution. It is intended to help users operate the system and integrate it into existing development environments.

The primary goal of this manual is to enable effective use of TestSplit through clear, step-by-step instructions. This includes guidance on executing commands, interpreting outputs, and integrating with CI platforms such as GitHub Actions and GitLab CI.

The manual presents the system's functionality in a structured, accessible manner, focusing on practical use to support quick adoption without requiring detailed knowledge of the underlying implementation.

1.2 Overview of TestSplit

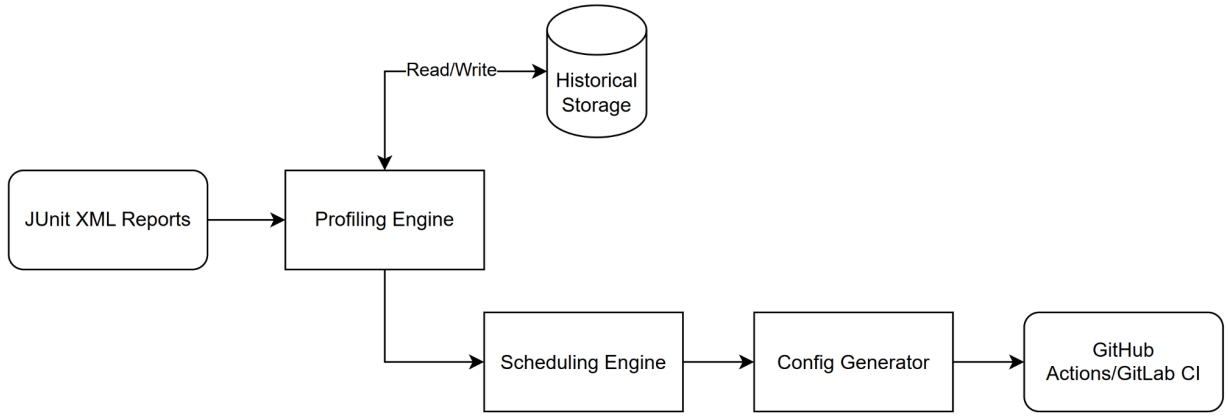


Figure 1: High-Level Overview of TestSplit

TestSplit optimises Continuous Integration (CI) pipelines by intelligently distributing test cases across parallel jobs. It addresses a common challenge in modern software development, where large and unevenly distributed test suites result in long execution times and delayed feedback.

The system analyses test execution data, typically obtained from JUnit XML reports, to determine the duration of individual test cases. Based on this data, TestSplit applies scheduling algorithms such as Longest Processing Time (LPT) by default, or MULTIFIT as an alternative, to partition tests into balanced groups. These groups are executed in parallel, reducing overall runtime by minimising the duration of the slowest job, known as the critical path.

In addition to test distribution, TestSplit provides features that support CI optimisation workflows. These include generating CI configuration files for platforms such as GitHub Actions and GitLab CI, executing

test subsets in parallel locally, and comparing historical profiling results to detect performance regressions. A lightweight dashboard is also available to visualise metrics such as execution time, workload balance, and predicted speed-up.

TestSplit integrates with existing development workflows and requires minimal configuration. It works with standard test output formats and does not require modifications to test code or repository structure. This allows development teams to improve CI efficiency and reduce feedback cycles with minimal disruption.

1.3 Target Users

TestSplit is intended for technically proficient users who work with automated testing and Continuous Integration (CI) pipelines. The primary users include software developers, DevOps engineers, QA engineers, and technical leads who are responsible for maintaining test infrastructure and optimising CI performance.

These users are typically familiar with version control systems, CI/CD platforms such as GitHub Actions and GitLab CI, and testing frameworks that generate JUnit-compatible reports. As TestSplit operates through a command-line interface, users are expected to have a basic understanding of terminal usage and software build tools (Maven or Gradle).

The user base can be broadly categorised into two groups:

- **Beginner Users**: Developers with limited experience in CI optimisation who require simple commands and clear guidance to profile test suites and generate optimised configurations. These users benefit from straightforward workflows and explanatory outputs.
- **Advanced Users**: Experienced developers and DevOps engineers who are comfortable customising CI pipelines and analysing performance metrics. These users may leverage advanced options such as alternative scheduling algorithms, variance-aware tuning, and runtime optimisation strategies to fine-tune test execution.

By accommodating both user groups, TestSplit is designed to be accessible for initial adoption while also providing sufficient depth and flexibility for advanced performance optimisation.

2. System Overview

2.1 What TestSplit Does

TestSplit improves the efficiency of Continuous Integration (CI) pipelines by reducing the total execution time of automated test suites. It achieves this by analysing test execution data and distributing tests across multiple parallel jobs in a balanced manner.

Test execution data, typically obtained from JUnit XML reports, is used to determine the duration of individual test cases. Based on this information, TestSplit groups tests so that each parallel job has a similar workload. This reduces the duration of the slowest job, known as the critical path, which determines the overall runtime of the CI test stage.

In addition to distributing tests, TestSplit automates key steps in the CI optimisation workflow. It can generate CI configuration files for platforms such as GitHub Actions and GitLab CI, enabling parallel execution without manual configuration. It also supports local execution of test subsets, allowing validation of performance improvements before integration into CI pipelines.

TestSplit also provides tools for analysing performance over time. Profiling data from previous runs can be stored and compared to detect regressions and evaluate optimisation effectiveness. A dashboard is available to visualise metrics such as execution time, workload balance, and predicted speed-up.

2.2 Main Components

TestSplit is composed of several interconnected components that analyse test execution data, optimise distribution, and support integration with CI workflows. Each component performs a specific role within the system, forming a cohesive pipeline from input data to optimised output.

2.2.1 Command Line Interface

The Command Line Interface (CLI) is the primary point of interaction with TestSplit. It enables execution of all core functionalities through a set of structured commands.

Through the CLI, test suites can be profiled, CI configurations generated, tests executed in parallel, historical runs compared, configuration files validated, and the dashboard launched. Each command supports a range of options that allow behaviour to be customised, such as selecting scheduling algorithms, adjusting job counts, or enabling advanced execution modes.

The CLI is designed to be both accessible and extensible. Simple commands with minimal configuration support initial use, while optional flags and parameters allow more advanced control. Built-in help commands provide guidance on usage and available options, enabling quick understanding of how to interact with the system.

2.2.2 Profiling Engine

The Profiling Engine is responsible for analysing test execution data and constructing a performance profile of the test suite. It processes input data, typically in the form of JUnit XML reports, and extracts key information, including individual test durations, total execution time, and the number of test cases.

Using this data, the Profiling Engine computes summary statistics, including average test duration and identifies important characteristics such as long-running or zero-duration tests. These metrics form the

foundation for subsequent scheduling and optimisation, as they provide an accurate representation of how the test suite behaves in practice.

In addition to generating a profile for a single run, the Profiling Engine also contributes to historical analysis by producing structured data that can be stored and reused across multiple executions. This enables TestSplit to track performance trends over time and support regression detection.

2.2.3 Scheduling Engine

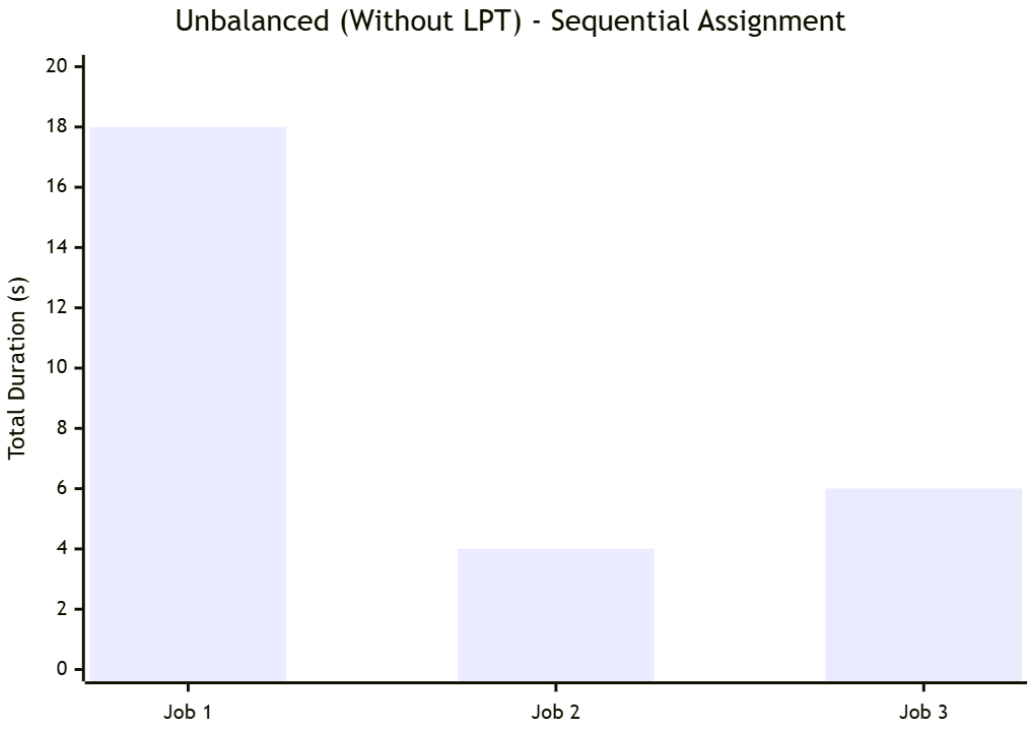


Figure 2: Sequential Assignment of Jobs (Without LPT)

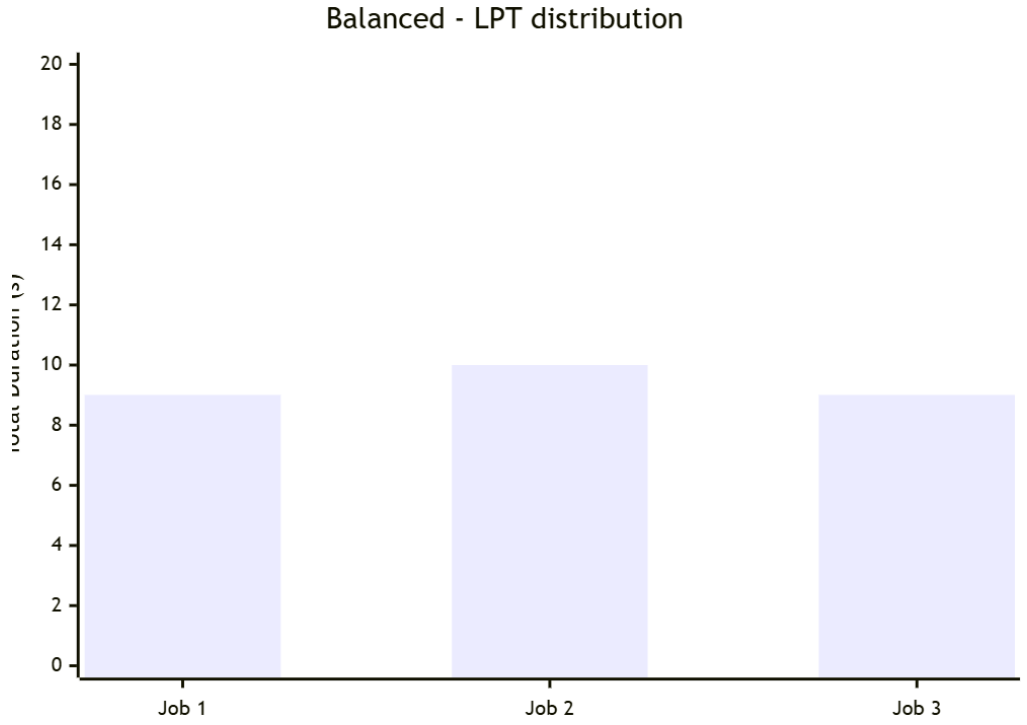


Figure 3: LPT-based Distribution of Jobs

The Scheduling Engine distributes test cases across multiple parallel jobs to minimise total execution time. It uses performance data from the Profiling Engine to group tests, ensuring each job has a balanced workload.

By default, the Longest Processing Time (LPT) algorithm is applied. This approach sorts tests by execution time in descending order and assigns each test to the job with the lowest current workload, helping prevent long-running tests from being concentrated in a single job and reducing bottlenecks.

As an alternative, the MULTIFIT algorithm is also supported. This method iteratively adjusts job assignments to better approximate an optimal distribution. The desired algorithm can be selected through CLI options based on optimisation requirements.

Additional factors, such as variability in test execution time, can be incorporated through configurable parameters, such as the risk factor. This allows the system to account for unstable or inconsistent tests when generating distributions.

The output is a set of job groups, each containing a subset of tests with an associated total execution time. These groups are then used to generate CI configurations or enable parallel execution.

2.2.4 Configuration Generator

TestSplit generates CI configuration files that implement the optimised test distribution, enabling parallel test execution without requiring manual changes to existing pipelines. It takes computed job groupings and converts them into executable workflows for supported platforms such as GitHub Actions and GitLab CI.

Rather than requiring users to rewrite their CI configuration, TestSplit works with existing files and applies the necessary modifications to enable parallel execution. It identifies relevant test jobs, extracts existing test commands, and replaces them with multiple parallel jobs that each execute a subset of the test suite.

The generated configuration can be customised through various options, including the number of jobs, runner configuration, test commands, and output file location. A dry-run mode is also available, allowing the configuration to be previewed before being written to a file.

2.2.5 Historical Storage

TestSplit stores profiling data across multiple executions, allowing users to track and analyse test performance over time. This data includes key metrics such as total test duration, average duration, critical path, and workload balance.

By maintaining a history of profiling runs, users can compare results across different executions and identify trends or regressions in test performance. For example, an increase in critical path duration or a higher imbalance between jobs may indicate inefficiencies in the test suite.

This stored data is used by features such as the compare command and the dashboard, which rely on historical records to provide insights into performance changes. The data is saved locally in a designated directory, ensuring that results persist between runs without requiring external services or additional setup.

2.2.6 Dashboard

The Dashboard allows users to visually analyse and interpret the performance data generated by TestSplit. It provides a clear and accessible way to understand how a test suite behaves over time.

Using the dashboard, users can view key metrics such as total execution time, critical path, workload balance, and predicted speed-up. It also allows users to compare multiple profiling runs, making it easier to identify performance regressions or improvements and evaluate the effectiveness of different scheduling strategies.

The dashboard is accessed through a locally hosted web interface. It reads data generated by the CLI and does not require external services or databases, making it simple to set up and use within a development environment.

2.3 Supported Workflow

TestSplit supports an end-to-end workflow that integrates into existing development and Continuous Integration (CI) processes. The workflow begins by running a test suite using an existing build or test framework, which generates JUnit XML reports containing execution details for each test case.

These reports are then analysed by TestSplit to extract execution times and build a performance profile of the test suite. Based on this profile, tests are distributed into balanced groups so that each parallel job has a similar workload, reducing total execution time.

Once the distribution is computed, an optimised CI configuration can be generated. This configuration modifies the existing pipeline by introducing parallel jobs that execute subsets of the test suite. Alternatively, the distributed test groups can be executed locally to validate performance improvements before applying changes to the CI pipeline.

After execution, performance data is stored to track changes over time. Results can then be analysed using the dashboard or compared across runs using CLI commands to identify regressions or improvements.

2.4 Test Distribution and Optimisation

TestSplit optimises test execution by distributing test cases across parallel jobs to reduce overall pipeline runtime. The objective is to minimise the duration of the slowest job, known as the critical path, while keeping workload distribution as balanced as possible across all jobs.

This optimisation is based on measured test execution data. By analysing historical test durations, TestSplit determines how tests should be grouped to achieve more efficient parallel execution. Several scheduling strategies are supported, each offering different trade-offs between simplicity, balance, and computational cost.

TestSplit also supports both static and runtime-aware optimisation approaches. Static scheduling uses historical profiling data to generate balanced job groups before execution begins, while optional runtime strategies can further improve utilisation during parallel execution.

2.4.1 Input Data Sources

TestSplit uses test execution reports as the primary input for profiling and distribution. These reports provide the data required to analyse test performance and determine how tests should be grouped for parallel execution.

The main supported input format is JUnit XML. These reports contain structured information about each test case, including its name, execution time, and result status. This allows TestSplit to build an accurate performance profile of the test suite.

JUnit XML is widely supported by common testing frameworks such as JUnit and can also be generated by other frameworks, including TestNG, provided they produce compatible output. This ensures that TestSplit can be used across a range of testing environments without requiring changes to the test code.

Accurate and complete input data is essential for effective optimisation. Missing or inconsistent execution times may reduce the quality of the generated distribution. To ensure reliable results, test frameworks should be configured to produce valid and detailed test reports before running TestSplit.

2.4.2 Test Distribution Overview

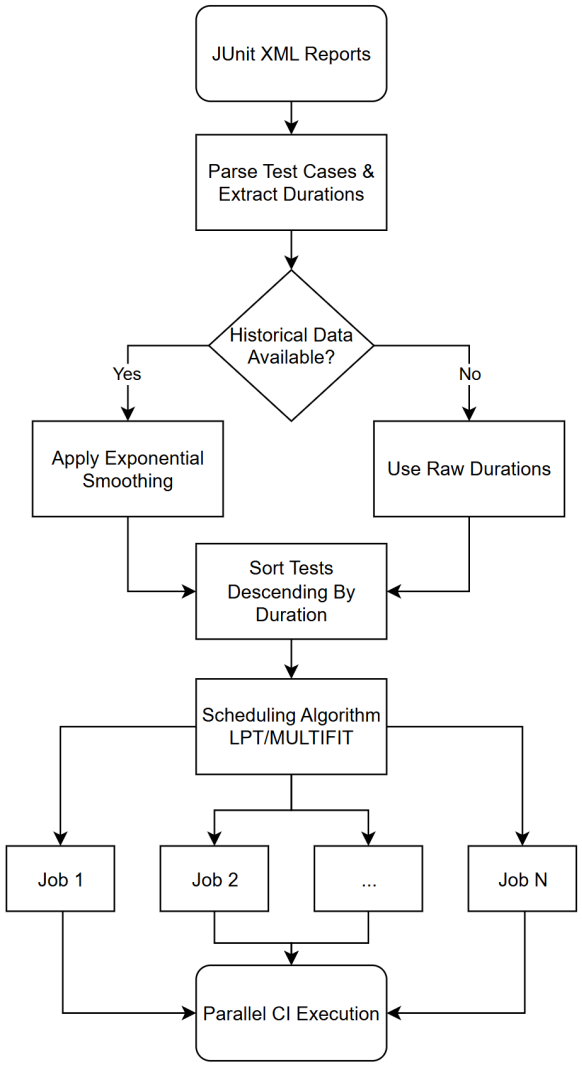


Figure 4: Test Distribution Flowchart

TestSplit distributes test cases across multiple parallel jobs to reduce overall execution time while maintaining a balanced workload. The distribution process uses historical test execution data to determine how tests should be grouped for efficient parallel execution.

Test cases are first analysed to obtain their execution durations. These durations are then used to assign tests to jobs in a way that avoids uneven workloads, where some jobs finish significantly earlier than others. By balancing the total execution time of each job, TestSplit minimises idle time and improves resource utilisation.

The distribution is performed using scheduling algorithms, such as Longest Processing Time (LPT) and MULTIFIT, which aim to produce near-optimal groupings based on the available data. These algorithms ensure that longer tests are distributed carefully to prevent bottlenecks and reduce the likelihood of a single job dominating the total runtime.

The result of this process is a set of job groups, each containing a subset of tests with a similar total execution time. These groups can then be executed in parallel, either within a CI pipeline or locally, resulting in faster feedback and more efficient test execution.

2.4.3 LPT Scheduling

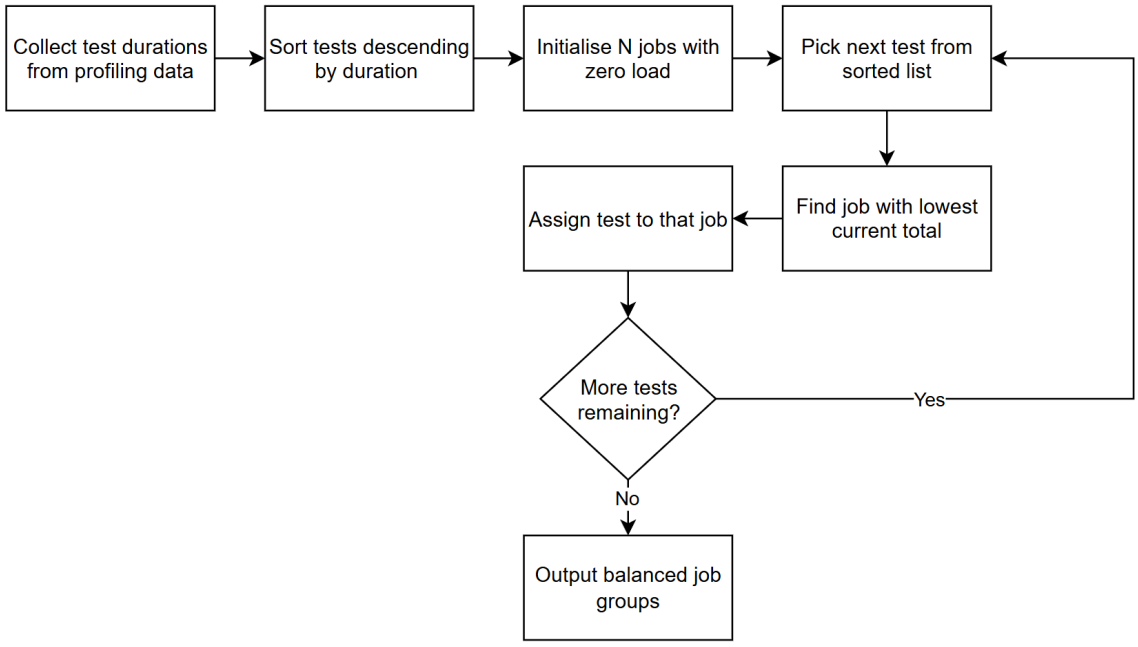


Figure 5: LPT-Scheduling Flowchart

The Longest Processing Time (LPT) algorithm is the default scheduling strategy used by TestSplit to distribute test cases across parallel jobs. It is designed to produce a balanced workload by prioritising longer-running tests during distribution.

In this approach, test cases are first sorted in descending order by execution duration. Each test is then assigned to the job with the lowest current total execution time. By placing longer tests first, the algorithm reduces the likelihood that one job becomes significantly slower than the others.

LPT is efficient and performs well in most scenarios, providing a good balance between simplicity and effectiveness. It is particularly suitable for test suites with relatively stable execution times, where historical durations are a reliable indicator of future performance.

While LPT does not guarantee a perfectly optimal distribution, it produces near-optimal results in practice with low computational overhead. For this reason, it is used as the default scheduling algorithm in TestSplit.

2.4.4 MULTIFIT Scheduling

MULTIFIT is an alternative scheduling algorithm supported by TestSplit that aims to produce a more balanced distribution of test cases across parallel jobs. It is designed to improve upon simpler approaches by more closely approximating an optimal test assignment.

Unlike LPT, which assigns tests greedily in a single pass, MULTIFIT uses an iterative process to refine job assignments. It repeatedly adjusts how tests are grouped based on an estimated target workload for each job, seeking to minimise the maximum execution time across all jobs.

MULTIFIT can achieve better balance in scenarios where test durations vary significantly or where a more precise distribution is required. However, this improved accuracy comes at the cost of greater computational complexity than LPT.

In practice, MULTIFIT is useful for larger or more variable test suites where achieving a tighter balance between jobs can lead to noticeable reductions in overall execution time. Users can select this algorithm through CLI options when higher optimisation accuracy is required.

2.4.5 Variance-Aware Scheduling

Variance-aware scheduling extends the standard test distribution by accounting for variability in test execution time. Instead of relying solely on the average duration, TestSplit adjusts each test's weight to reflect both its mean execution time and its variability.

This is achieved by applying a configurable risk factor to the standard deviation of each test’s duration. Tests with higher variability are treated as potentially more expensive, reducing the likelihood that they are grouped together in a way that could lead to unpredictable delays.

By incorporating variability into scheduling decisions, this approach helps produce more robust distributions, particularly for test suites with inconsistent or unstable execution times. It reduces the risk of underestimating the cost of certain tests and improves the reliability of parallel execution.

Variance-aware scheduling can be enabled through CLI options, allowing the level of risk adjustment to be tuned based on the characteristics of the test suite. This provides additional control over how aggressively TestSplit accounts for uncertainty during distribution.

2.4.6 Static vs Dynamic Optimisation (*Static vs Dynamic Scheduling)*

TestSplit supports both static and dynamic approaches to test distribution, allowing optimisation to be performed either before execution or during runtime.

Static optimisation is based on historical test execution data. TestSplit analyses previous test durations and generates a fixed distribution of test cases before execution begins. This approach is efficient and works well when test execution times are stable and predictable.

Dynamic optimisation adapts to runtime conditions during execution. Instead of relying solely on a predefined distribution, tasks can be reassigned as jobs complete or become idle. Features such as dynamic work queues and work stealing allow available resources to be utilised more effectively by redistributing remaining work across jobs.

Dynamic approaches are particularly useful when test execution times are unpredictable or vary significantly between runs. By adjusting workload distribution during execution, they help reduce idle time and improve overall resource utilisation.

In practice, static optimisation provides a reliable baseline, while dynamic strategies can further improve performance in more variable environments. The choice between these approaches depends on the characteristics of the test suite and the level of optimisation required.

3. Installation & Setup

This section describes the requirements and steps needed to set up TestSplit for use in a development environment. It covers system prerequisites, project setup, and input requirements necessary for successful execution.

3.1 Prerequisites

Before installing and using TestSplit, ensure that the following prerequisites are met:

- **Node.js** (version 16 or later)
    - Required to run the TestSplit CLI tool.
- **npm** or **yarn**
    - Used to install dependencies and manage the project.
- **Java build tool**
    - Maven or Gradle is used to run tests and generate JUnit XML reports.
- **JUnit-compatible test reports**

    - TestSplit requires test execution data in JUnit XML format. Ensure your test framework is configured to generate these reports.
- **CI environment (optional)**
    - GitHub Actions or GitLab CI is recommended if you intend to generate and use CI configurations.
- **Basic command-line knowledge**
    - Familiarity with terminal usage is required to run commands and configure options.

Meeting these prerequisites ensures that TestSplit can be installed and used effectively for profiling, scheduling, and CI optimisation.

3.2 Project Setup

To begin using TestSplit, clone the project repository and install the required dependencies.

```
git clone <repository-url>
cd testsplit
npm install
```

Once dependencies are installed, TestSplit can be executed using the CLI. You may use npx or link the package globally if preferred.

```
npx testsplit --help
```

This confirms that the tool is correctly installed and ready for use.

3.3 Input Requirements

TestSplit relies on specific inputs to perform profiling and optimisation. These inputs provide the necessary data to analyse test performance and generate distributions.

3.3.1 JUnit XML Reports

JUnit XML reports are the primary input used by TestSplit. These reports contain structured information about test cases, including execution time, status, and identifiers.

Ensure that your test framework is configured to generate JUnit XML output. For example, Maven and Gradle projects can be configured to produce reports in standard directories such as:

```
target/surefire-reports/
build/test-results/
```

Accurate and complete reports are essential for effective profiling and scheduling.

3.3.2 Existing CI Configuration

To generate optimised CI workflows, TestSplit requires an existing CI configuration file. This file serves as a base and is modified to introduce parallel test execution.

Supported platforms include:

- GitHub Actions
- GitLab CI

If no configuration file is found, it must be provided manually using CLI options.

3.3.3 Source Directory

The source directory is used to detect test dependencies and project structure. By default, TestSplit expects test sources to be located in standard directories such as:

```
src/test/java
```

This information is used to improve scheduling accuracy, particularly when handling test dependencies.

4. Command-Line Interface

4.1 CLI Overview

TestSplit is operated through a command-line interface that provides access to all core features. Commands are organised by functionality, allowing tasks such as profiling, configuration generation, execution, and analysis to be performed efficiently.

Each command includes a set of options that allow behaviour to be customised based on the requirements of the test suite and CI environment.

4.2 General Usage Format

TestSplit commands follow a consistent structure:

```
testsplit <command> [options]
```

For example:

```
testsplit profile --junit target/surefire-reports --jobs 4
```

Options are used to control behaviour, such as specifying input paths, selecting scheduling algorithms, or adjusting the number of parallel jobs.

4.3 Getting Help

Help information is available directly through the CLI.

To view all available commands:

```
testsplit --help
```

To view options for a specific command:

```
testsplit <command> --help
```

For example:

```
testsplit profile --help
```

This displays detailed information about available options and their usage, making it easier to understand how to interact with the tool.

5. User Guide

This section provides step-by-step instructions for using TestSplit's core commands. Each command is described with its purpose, usage, and example output.

5.1 Profiling a Test Suite (profile)

5.1.1 Purpose

The profile command analyses test execution data and generates a performance profile of the test suite. It provides insights into test duration, workload balance, and expected parallel performance.

This command is typically the first step when using TestSplit, as it produces the data required for scheduling and optimisation.

5.1.2 Basic Command

```
testsplit profile --junit <path-to-report>
```

5.1.3 Important Options

- **--junit <path>**: Path to a JUnit XML file or directory. If omitted, TestSplit attempts to auto-detect the report location.
- **--jobs <number>:** Number of parallel jobs to simulate. Defaults to the number of available CPU cores.
- **--algorithm <lpt|multifit>:** Scheduling algorithm used to distribute tests.
- **--risk-factor <number>**: Adjusts variance-aware scheduling by weighting test duration variability.
- **--explain**: Provides a plain English explanation of the profiling results.

5.1.4 Example

```
testsplit profile --junit target/surefire-reports --jobs 4 --algorithm lpt
```

5.1.5 Output Explanation

The profile command produces several key metrics:

- **Tests parsed:** Total number of test cases analysed.
- **Total duration:** Combined execution time of all tests.
- **Critical path:** The longest-running job after distribution. This determines total execution time.
- **Predicted speed-up:** Estimated improvement compared to sequential execution.
- **Balance ratio:** A measure of how evenly tests are distributed across jobs.
- **Job distribution:** Breakdown of tests assigned to each parallel job.

These metrics help identify inefficiencies in the test suite and evaluate how well tests can be parallelised

5.2 Generating CI Configuration (generate-config)

5.2.1 Purpose

The generate-config command creates an optimised CI configuration based on the profiled test distribution. It converts computed job groupings into parallel CI jobs, enabling efficient test execution across multiple runners.

This command automates the modification of existing CI pipelines, eliminating the need to manually configure parallel test execution.

5.2.2 Basic Command

```
testsplit generate-config --junit <path-to-report>
```

5.2.3 Important Options

- **--junit <path>:** Path to JUnit XML reports. If omitted, TestSplit attempts to auto-detect the report location.

- **--jobs <number>:** Number of parallel jobs to generate.
- **--platform <github|gitlab>:** Target CI platform. Defaults to GitHub Actions.
- **--out <file>:** Output file path for the generated CI configuration.
- **--template <path> or --from <path>-out <file>:** Path to an existing CI configuration file to use as a base.
- **--test-command <command>**: Overrides the detected test command used in CI jobs.
- **--runner-cores <number>:** Number of CPU cores per CI runner, used to optimise job grouping.
- **--algorithm <lpt|multifit>**: Scheduling algorithm used to distribute tests.
- **--risk-factor <number>**: Adjusts variance-aware scheduling.
- **--dry-run**: Outputs the generated configuration to the console without writing to a file.

5.2.4 Example (GitHub)

```
testsplit generate-config \
 --junit target/surefire-reports \
 --platform github \
 --jobs 4 \
 --out .github/workflows/newcifile.yml
```

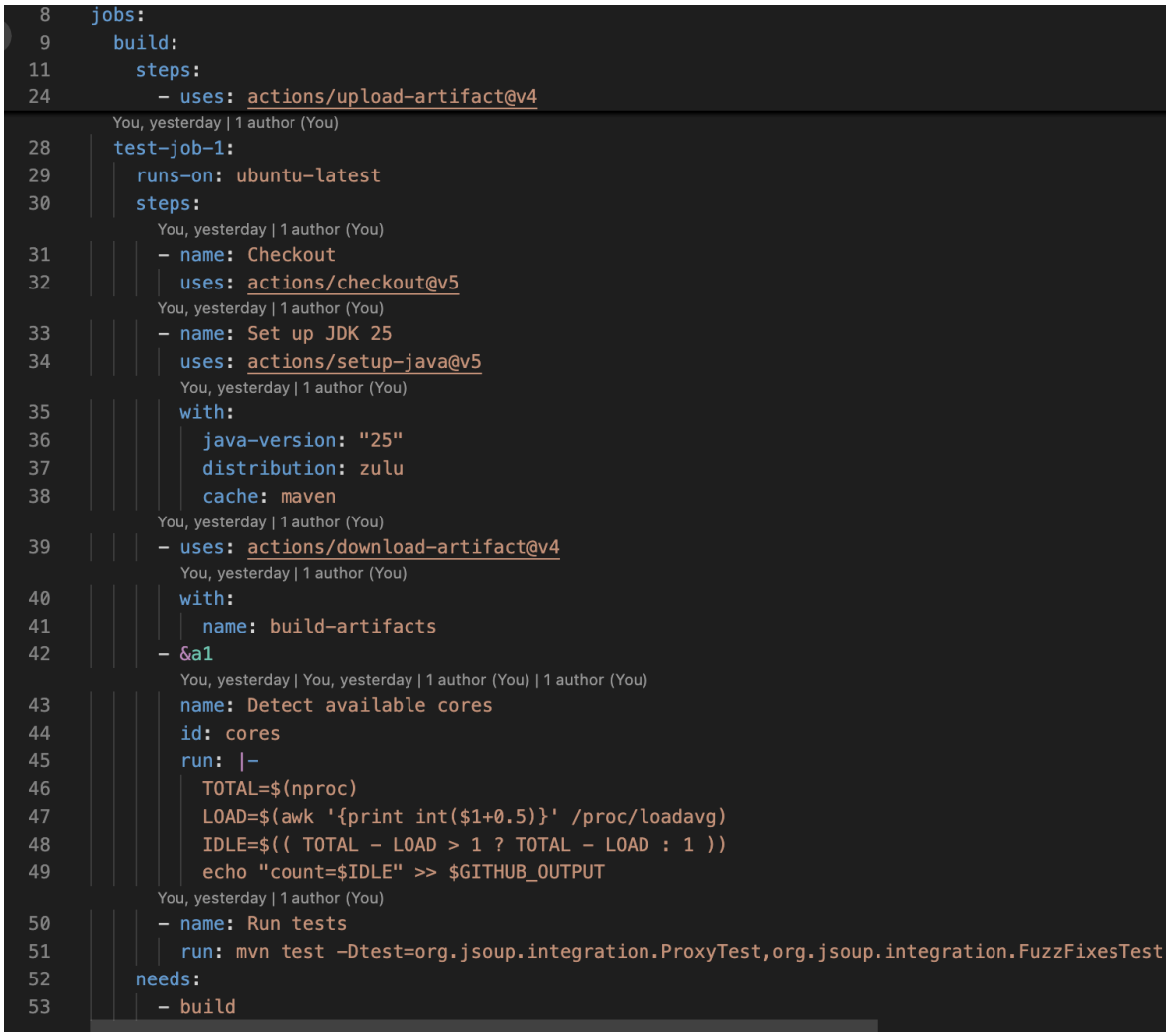
This generates a GitHub Actions workflow with parallel test jobs.

5.2.5 Example (GitLab)

```
testsplit generate-config \
--junit target/surefire-reports \
--platform gitlab \
--jobs 4 \
--out .gitlab-ci.yml
```

Figure 6: JSoup CI Pipeline

5.2.6 Dry Run Mode



```
jobs:
  build:
    steps:
      - uses: actions/upload-artifact@v4
  You, yesterday | 1 author (You)
  test-job-1:
    runs-on: ubuntu-latest
    steps:
      You, yesterday | 1 author (You)
      - name: Checkout
        uses: actions/checkout@v5
      You, yesterday | 1 author (You)
      - name: Set up JDK 25
        uses: actions/setup-java@v5
        You, yesterday | 1 author (You)
        with:
          java-version: "25"
          distribution: zulu
          cache: maven
      You, yesterday | 1 author (You)
      - uses: actions/download-artifact@v4
        You, yesterday | 1 author (You)
        with:
          name: build-artifacts
      - &a1
        You, yesterday | You, yesterday | 1 author (You) | 1 author (You)
        name: Detect available cores
        id: cores
        run: |-
          TOTAL=$(nproc)
          LOAD=$(awk '{print int($1+0.5)}' /proc/loadavg)
          IDLE=$(( TOTAL - LOAD > 1 ? TOTAL - LOAD : 1 ))
          echo "count=$IDLE" >> $GITHUB_OUTPUT
      You, yesterday | 1 author (You)
      - name: Run tests
        run: mvn test -Dtest=org.jsoup.integration.ProxyTest,org.jsoup.integration.FuzzFixesTest
    needs:
      - build
```

Figure 7: JSoup Generated CI Pipeline File

Dry run mode allows the generated CI configuration to be previewed without writing it to a file.

```
testsplit generate-config --junit target/surefire-reports --dry-run
```

This is useful for verifying the generated configuration before applying changes to the repository.

5.2.7 Auto-Detection Behaviour

If the --junit option is not provided, TestSplit attempts to automatically detect the location of test reports by scanning common directories.

Similarly, if a CI configuration file is not explicitly specified, TestSplit attempts to locate an existing configuration file based on the selected platform.

If auto-detection fails, the required paths must be provided manually using CLI options.

5.3 Comparing Historical Runs (compare)

5.3.1 Purpose

The compare command analyses historical profiling data to identify changes in test performance over time. It compares recent runs and highlights differences in key metrics, including total duration, critical path, and workload balance.

This command is useful for detecting performance regressions, evaluating the impact of changes to the test suite, and monitoring trends across multiple executions. It enables more informed decisions when optimising test distribution and CI performance.

5.3.2 Basic Command

```
testsplit compare
```

5.3.3 Important Options

- **--runs <number>:** Number of recent runs to compare. Defaults to 2.
- **--threshold <percentage>:** Percentage threshold used to detect regressions. Defaults to 10%.

5.3.4 Example

```
testsplit compare --runs 3 --threshold 15
```

This compares the three most recent profiling runs and flags regressions exceeding 15%.

5.3.5 Output Explanation

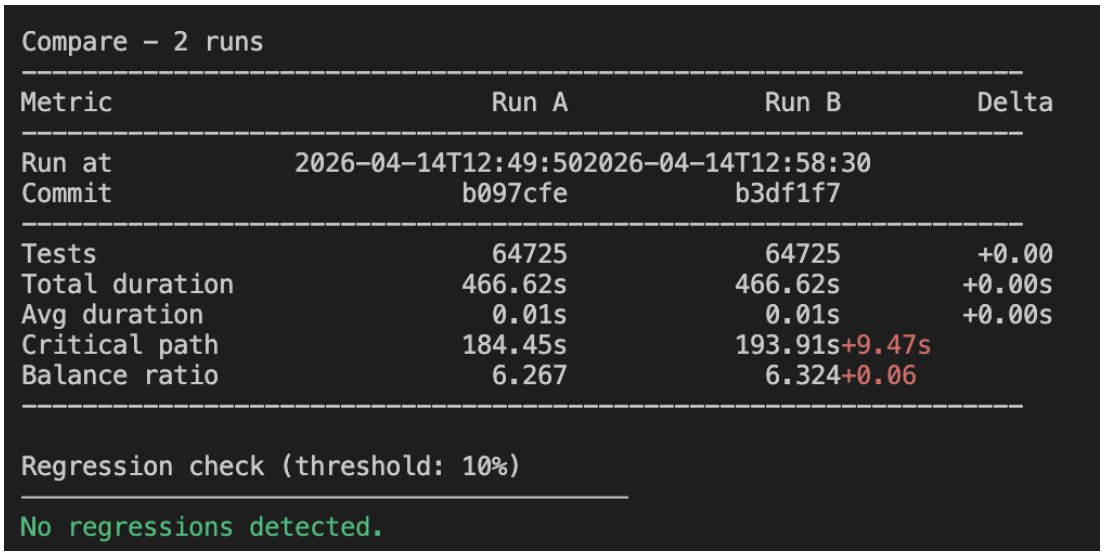


Figure 8: Compare Command Results

The compare command outputs a comparison table of key metrics across runs, including:

- **Test count:** Number of tests executed in each run.
- **Total duration:** Overall execution time of the test suite.
- **Average duration:** Average execution time per test.
- **Critical path:** Duration of the longest-running job.
- **Balance ratio:** A measure of how evenly tests are distributed.

Each metric includes a delta value indicating the change between runs. Improvements are typically highlighted in green, while regressions are shown in red.

A regression check is also performed based on the specified threshold. If any metric exceeds this threshold, it is flagged as a regression, helping to quickly identify performance issues.

5.4 Running a Benchmark (benchmark)

5.4.1 Purpose

The benchmark command measures the expected performance improvement gained from parallelising test execution. It compares the total execution time of the test suite when run sequentially against the predicted execution time when tests are distributed across multiple parallel jobs.

This command helps evaluate the effectiveness of the scheduling strategy by quantifying time savings, speed-up, and workload balance. It is particularly useful for assessing whether parallelisation will yield meaningful improvements before implementing changes in a CI pipeline.

5.4.2 Basic Command

```
testsplit benchmark --junit <path-to-report>
```

This command analyses the provided JUnit XML reports and generates a benchmark comparing sequential and parallel execution performance.

5.4.3 Output Explanation

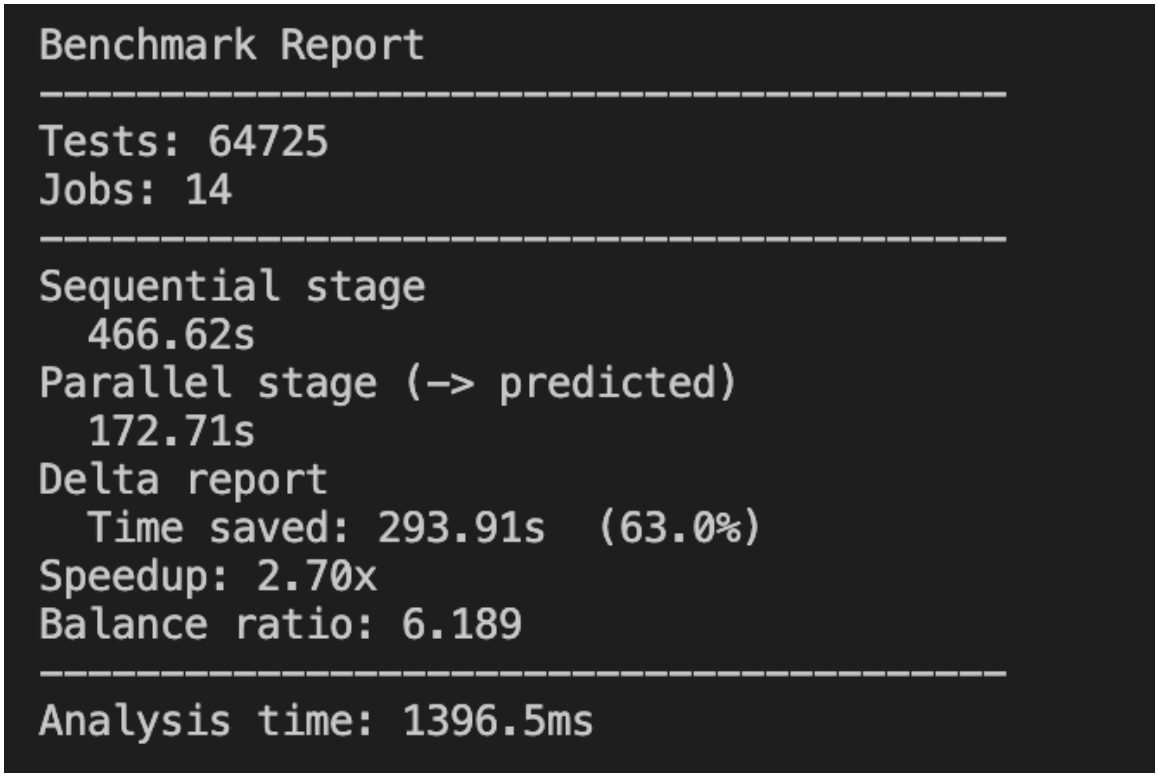


Figure 9: Benchmark Command Results

The benchmark command produces a summary of performance metrics, including:

- **Tests:** Total number of test cases analysed.
- **Sequential stage:** Total execution time when tests are run one after another.
- **Parallel stage:** Predicted execution time after distributing tests across parallel jobs.
- **Time saved:** Difference between sequential and parallel execution time.
- **Speed-up:** Ratio of performance improvement achieved through parallelisation.
- **Balance ratio:** A measure of how evenly tests are distributed across jobs.
- **Analysis time:** Time taken by TestSplit to compute the benchmark.

These metrics provide a clear indication of how effective parallel execution is for the given test suite.

5.5 Validating CI Configuration (validate)

5.5.1 Purpose

The validate command checks whether a CI configuration file is correctly structured and compatible with the selected platform. It ensures that required fields and execution steps are present before the configuration is used in a CI pipeline.

This helps prevent runtime errors caused by invalid or incomplete configurations and provides early feedback during setup.

5.5.2 Basic Command

```
testsplit validate --file <path-to-config>
```

5.5.3 Example

```yaml
# Scheduling settings used for this distribution
# algorithm: lpt
# risk_factor: 1
name: Build
on:
  push: null
  pull_request: null
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v5
      - name: Set up JDK 25
        uses: actions/setup-java@v5
        with:
          java-version: "25"
          distribution: zulu
          cache: maven
      - uses: actions/download-artifact@v4
        with:
          name: build-artifacts
      - name: Detect available cores
        id: cores
        run: |-
          TOTAL=$(nproc)
          LOAD=$(awk '{print int($1+0.5)}' /proc/loadavg)
          IDLE=$(( TOTAL - LOAD > 1 ? TOTAL - LOAD : 1 ))
          echo "count=$IDLE" >> $GITHUB_OUTPUT
      - name: Run tests
        run: mvn test
          -Dtest=org.jsoup.integration.ProxyTest,org.jsoup.integration.FuzzFixesTest
          -DfailIfNoSpecifiedTests=false -Dsurefire.forkCount=${{
          steps.cores.outputs.count }} -Dsurefire.reuseForks=true -DskipTests
          -Drat.skip=true
      - uses: actions/upload-artifact@v4
        with:
          name: build-artifacts
          path: target/
    needs:
      - build
    container: eclipse-temurin:21-jdk
```

Figure 10: Generated CI File (testsplit-ci.yml)

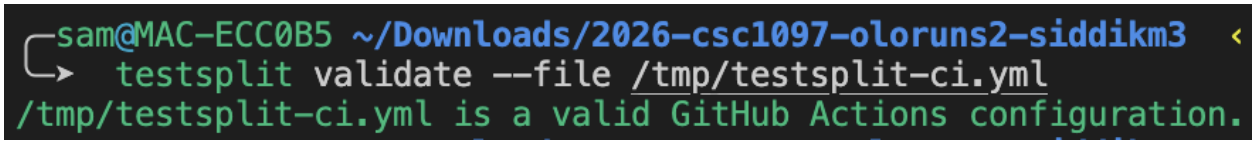


Figure 11: Validate Command Example

```
testsplit validate --file /tmp/testsplit-ci.yml
```

This validates a GitHub Actions configuration file generated by TestSplit.

5.5.4 Validation Rules

Validation checks include:

- Presence of required top-level fields (e.g. jobs for GitHub, stages for GitLab)
- Each job contains executable steps or scripts
- Configuration structure matches the selected platform
- YAML syntax is valid and correctly formatted

If any issues are detected, TestSplit reports them with clear error messages to help identify and resolve the problem.

5.6 Running Tests in Parallel (run)

5.6.1 Purpose

The run command executes test subsets in parallel based on the computed test distribution. It allows the effectiveness of scheduling strategies to be validated in a local environment before applying changes to a CI pipeline.

This command provides real execution results, including actual runtime per job, enabling comparison between predicted and observed performance.

5.6.2 Basic Command

```
testsplit run --junit <path-to-report> --cmd "<test-command>"
```

5.6.3 Important Options

- **--junit <path>:** Path to the JUnit XML report used for scheduling.
- **--jobs <number>**: Number of parallel jobs to execute.
- **--cmd <command>:** Base command used to run tests (e.g. mvn test -Dtest or npx jest).
- **--filter-flag <flag>**: Flag used to pass test filters to the test runner.
- **--filter-join <separator>**: Separator used to combine multiple test names.

- **--algorithm <lpt|multifit>**: Scheduling algorithm used to distribute tests.
- **--risk-factor <number>**: Adjusts variance-aware scheduling.
- **--dynamic**: Enables dynamic task assignment.
- **--steal**: Enables work stealing between jobs.
- **--affinity**: Enables CPU affinity for worker processes.

5.6.4 Example

```
testsplit run --junit target/surefire-reports --cmd "mvn test -Dtest"
```

This executes test subsets in parallel using the computed distribution.

5.6.5 Advanced Execution Modes

5.6.5.1 Dynamic Work Queue (--dynamic)

In dynamic mode, workers request new tasks when they become idle, rather than waiting for all assigned tasks to complete. This improves resource utilisation when test durations vary significantly.

5.6.5.2 Work Stealing (--steal)

Work stealing allows idle workers to take tasks from the busiest worker. This helps rebalance workloads during execution and reduces the impact of uneven test durations.

5.6.5.3 CPU Affinity (--affinity)

CPU affinity assigns each worker to a specific CPU core. This can improve execution consistency and reduce context switching, particularly on multi-core systems.

While not always necessary, this option can provide performance benefits in environments where CPU scheduling variability affects execution time.

5.6.6 Output Explanation

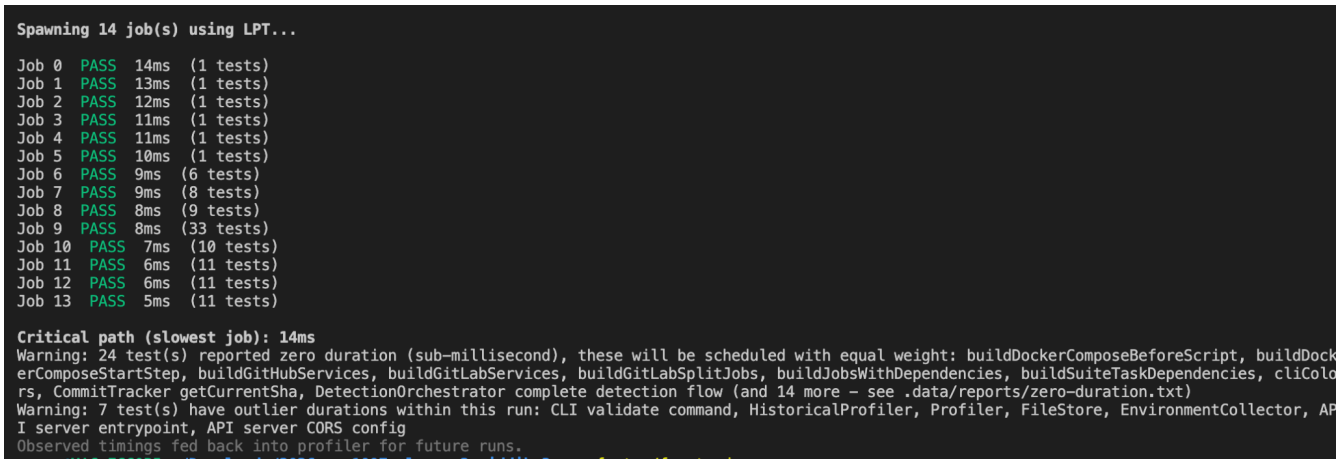


Figure 12: Run Command Example

The run command outputs execution results for each job, including:

- **Job ID:** Identifier for each parallel job.
- **Status:** Whether the job passed or failed.
- **Execution time:** Actual wall-clock time for each job.
- **Test count:** Number of tests executed per job.
- **Critical path:** The slowest job, representing the total execution time.

These results provide insight into real-world performance and help validate the effectiveness of the scheduling strategy.

5.7 Generating a Dockerfile (dockerfile)

5.7.1 Purpose

The Dockerfile command generates a Dockerfile for running tests in a containerised environment. This ensures a consistent and reproducible setup across different machines and CI systems.

It is particularly useful when tests require specific dependencies, environments, or system configurations that need to be standardised.

5.7.2 Basic Usage

```
testsplit dockerfile
```

This command generates a Dockerfile based on the detected project configuration, including the appropriate build tool and dependencies.

The generated Dockerfile can then be used to build a container image and run tests in an isolated environment.

5.8 Launching the Dashboard (dashboard)

5.8.1 Purpose

The dashboard command starts a local web-based interface for visualising test performance and analysing historical profiling data. It provides an accessible way to interpret metrics and understand how test distribution affects execution over time.

5.8.2 Basic Command

```
testsplit dashboard
```

5.8.3 Important Options

```
--port <number>
```

Specifies the port on which the dashboard server will run. Defaults to 3001.

```
--no-open
```

Starts the dashboard without automatically opening a browser window.

5.8.4 Example

```
testsplit dashboard --port 3001
```

This starts the dashboard on port 3001 and opens it in the default browser (unless disabled).

5.8.5 Dashboard Features

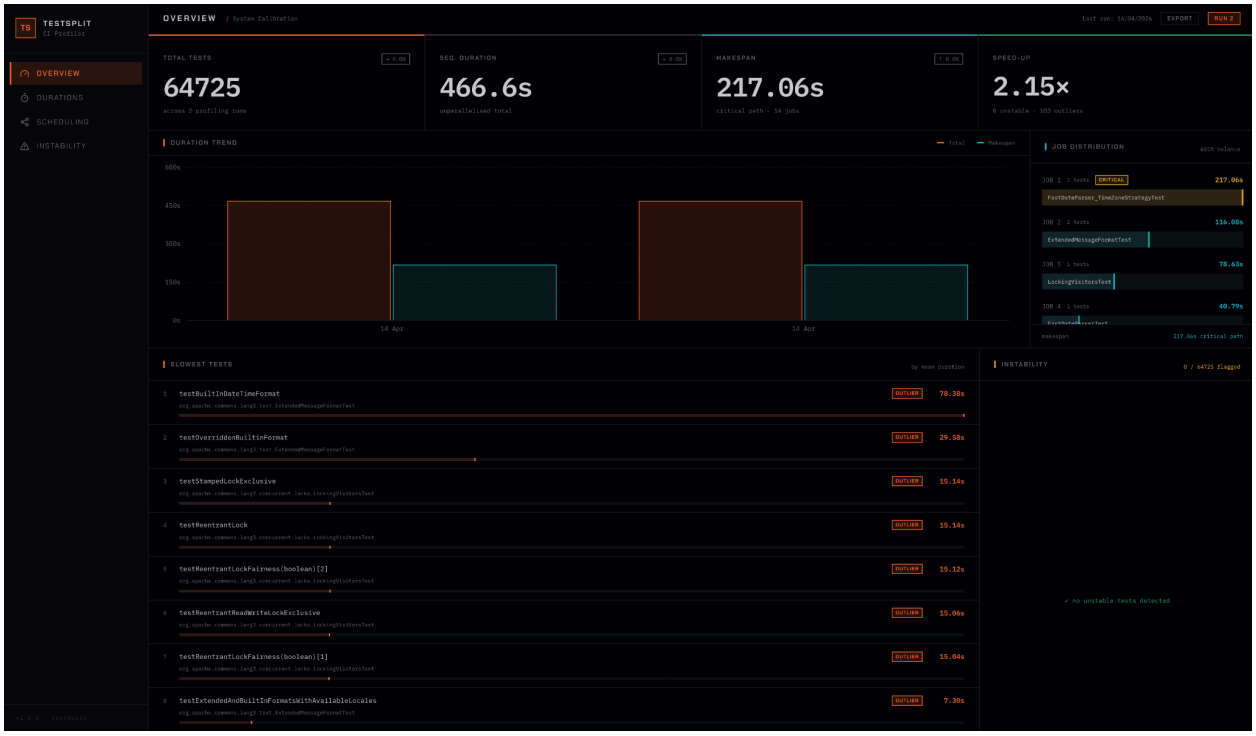


Figure 13: Overview Dashboard Page

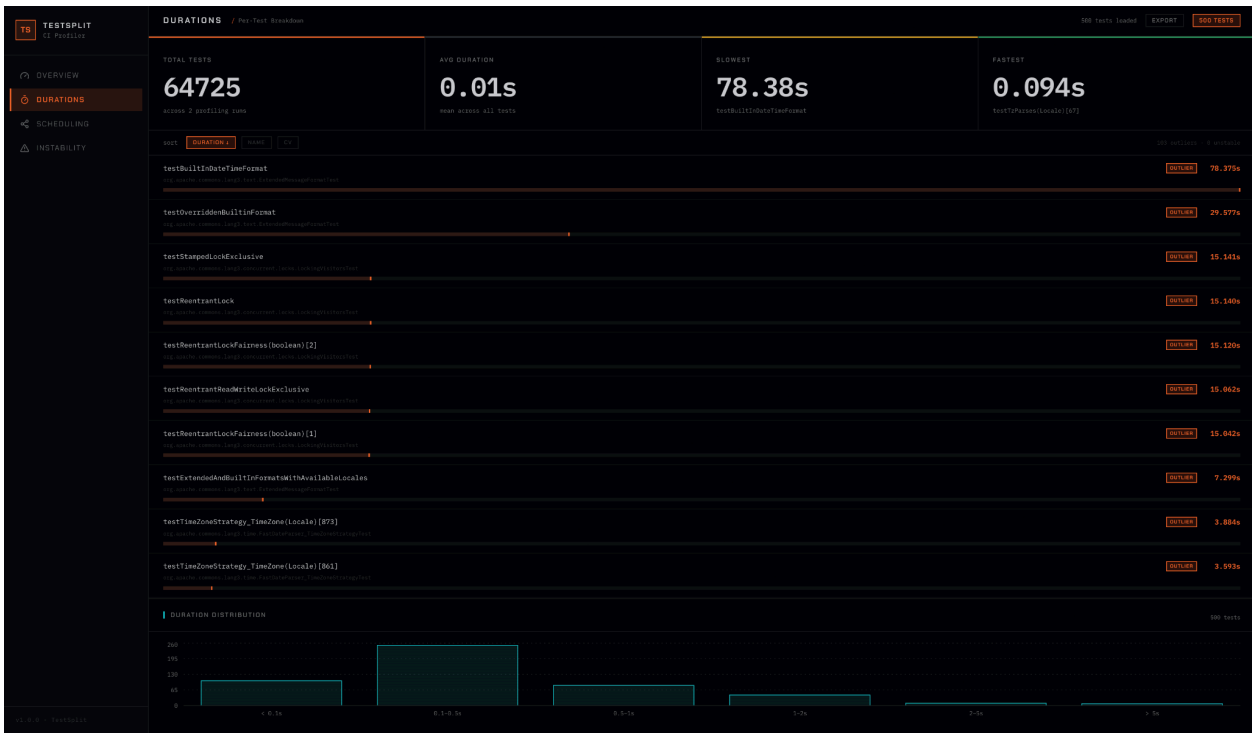


Figure 14: Durations Dashboard Page

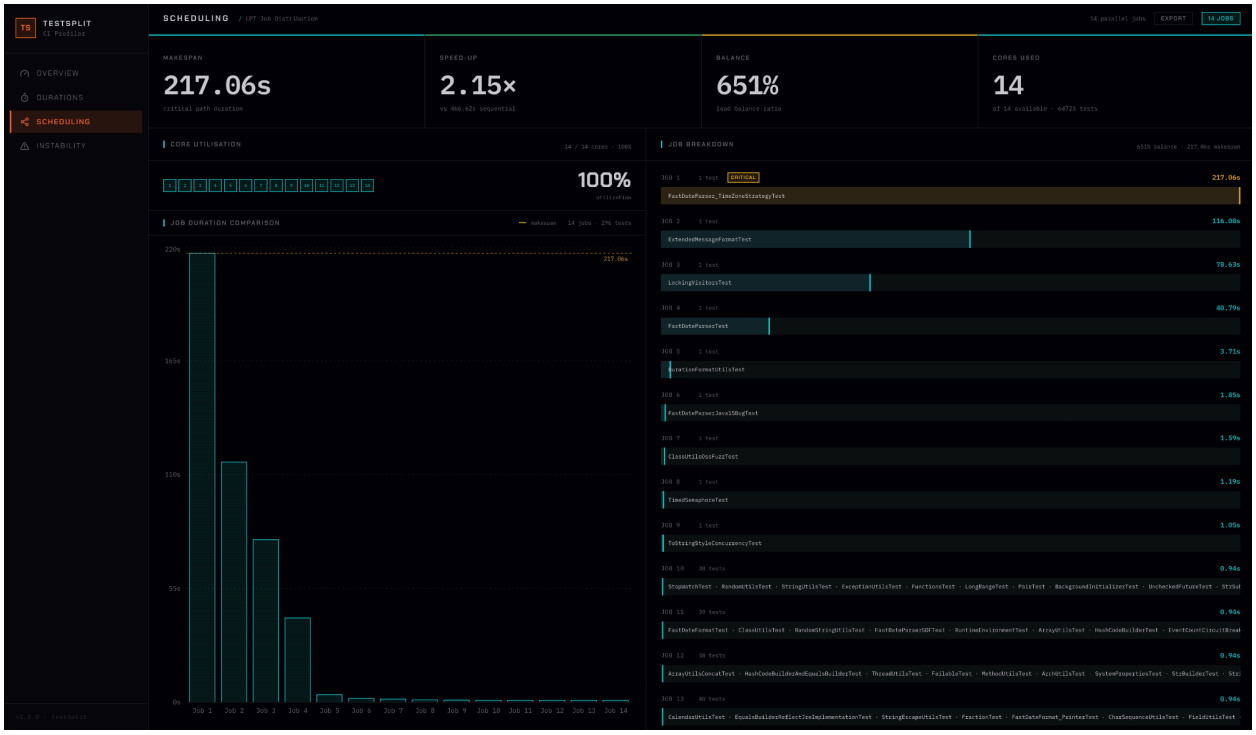


Figure 15: Scheduling Dashboard Page

Figure 16: Instability Dashboard Page

The dashboard provides:

- **Performance metrics visualisation:** Displays key metrics, including total execution time, critical path, and balance ratio.
- **Historical comparison:** Allows comparison of multiple profiling runs to identify regressions or improvements.
- **Workload distribution insights:** Shows how tests are distributed across jobs.
- **Trend analysis:** Enables tracking of performance changes over time.

These features help users better understand test behaviour and make informed optimisation decisions.

6. Example Workflow

This section demonstrates a typical workflow for using TestSplit to optimise test execution in a CI pipeline. It outlines the main steps from profiling to analysis.

6.1 Profiling a Project

Begin by profiling the test suite to analyse execution performance and generate scheduling data.

```
testsplit profile --junit <path-to-report>
```

This step produces key metrics such as total duration, critical path, and workload distribution, which are used for optimisation.

6.2 Generating CI Configuration

Use the profiling data to generate an optimised CI configuration with parallel test jobs.

```
testsplit generate-config --junit <path-to-report>
```

This creates a CI configuration file that distributes tests across multiple jobs, reducing overall execution time.

6.3 Comparing Runs

After multiple profiling runs, compare results to detect performance changes.

```
testsplit compare
```

This helps identify regressions or improvements in execution time and workload balance.

6.4 Using the Dashboard

Launch the dashboard to visualise performance metrics and analyse trends.

```
testsplit dashboard
```

The dashboard provides a graphical view of execution data, making it easier to interpret results and evaluate the effectiveness of optimisation.

7. Troubleshooting

This section outlines issues that may occur when using TestSplit and provides guidance on how to resolve them.

7.1 No JUnit Report Found

Issue: TestSplit cannot locate any JUnit XML reports.

Solution:

- Ensure the correct path is provided using the --junit option
- Verify that reports exist in standard directories (e.g. target/surefire-reports)
- Check that the test framework is configured to generate JUnit XML output

7.2 No Test Cases Parsed

Issue: JUnit reports are found, but no test cases are detected.

Solution:

- Ensure the reports contain valid test data
- Check that tests have been executed successfully
- Verify that execution times are included in the report

7.3 No Existing CI Config Found

Issue: TestSplit cannot find a CI configuration file.

Solution:

- Provide a configuration file using --from or --template
- Ensure the file exists and is accessible
- Confirm that the correct platform (GitHub or GitLab) is specified

7.4 Validation Failed

Issue: The generated CI configuration fails validation.

Solution:

- Check for missing required fields (e.g. jobs, steps, stages)
- Ensure all jobs contain executable commands
- Verify that the YAML syntax is correct

7.5 Port Already In Use

Issue: The dashboard cannot start because the port is already in use.

Solution:

- Use a different port with the --port option
- Stop the process currently using the port

7.6 Insufficient Historical Runs

Issue: The compare command cannot run due to insufficient data.

Solution:

- Run the profile command multiple times to generate historical data
- Ensure profiling data is being stored correctly

7.7 Dependency Cycle Detected

Issue: A circular dependency is detected between tests.

Solution:

- Review test dependencies (e.g. @DependsOnMethods, ordering annotations)
- Remove or refactor circular dependencies
- Ensure test execution order does not create cycles

8. Tips and Best Practices

This section provides practical recommendations for using TestSplit effectively and achieving optimal results when optimising test execution.

8.1 Start with Profiling

Always begin by running the profile command to analyse the test suite. Profiling provides the data required for accurate scheduling and highlights potential inefficiencies such as long-running or imbalanced tests.

8.2 Validate Before Committing

Before applying generated CI configurations, use the validate command to ensure the configuration is correct. This helps prevent pipeline failures caused by invalid or incomplete CI files.

8.3 Use Compare for Regression Testing

Run the compare command regularly to monitor changes in test performance. This allows regressions to be detected early and helps maintain consistent CI execution times.

8.4 Use Dashboard for Trend Analysis

Use the dashboard to visualise performance metrics and track trends over time. This provides a clearer understanding of how test distribution evolves and supports better optimisation decisions.

9. Conclusion

TestSplit provides a practical and efficient solution for optimising test execution in Continuous Integration (CI) pipelines. By analysing test performance data and distributing test cases across parallel jobs, it reduces overall execution time and improves feedback cycles for development teams.

Through its command-line interface, TestSplit enables users to profile test suites, generate optimised CI configurations, execute tests in parallel, and analyse performance over time. Additional features, such as historical comparisons and a visual dashboard, further support performance monitoring and continuous optimisation.

The tool is designed to integrate seamlessly into existing workflows with minimal configuration, making it accessible for initial adoption while still offering advanced features for fine-tuning performance. As a result, TestSplit helps teams improve CI efficiency, better utilise resources, and maintain scalable testing practices.